ШИНЖЛЭХ УХААН, ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ



БИЕ ДААЛТЫН ГАРЫН АВЛАГА

Үйлдлийн систем (F.CSM302): V-1-14

Төслийн нэр:	Contiguous Memory Allocation (Memory layout)
Шалгасан багш:	Г.Ганбат /F.CS10/
Бие даалтын ажил гүйцэтгэсэн:	B241910016 Я.Мөнх-Оргил B232290038 Н.Гантогтох B222270065 Э.Эрдэнэбат B241910045 Б.Баасандорж B242270080 Э.Халиунаа

Улаанбаатар хот, 2026.

Агуулга

ХЭСЭГ 1. ХЭРЭГЛЭГЧИЙН ЗААВАР 3
1.1 Програмын зорилго 3
1.2 Дэмжих орчин 3
1.3 Суулгах эсвэл нээх алхам 3
1.4 Ажиллуулах, зогсоох, reset хийх 3
1.5 Оролт ба тохиргоо 3
1.6 Дүрслэлийн тэмдэг, өнгө, төлөв (GUI) 4
1.7 Жишээнүүд 4
1.8 Түгээмэл алдаа ба шийдэл 5
ХЭСЭГ 2. OPERATIONAL ЗААВАР 6
2.1 Repository clone хийх 6
2.2 OS, runtime, version 6
2.3 Dependency 6
2.4 Build ба Run команд 6
2.5 Үндсэн тестийн өгөгдөл 6
2.6 Нэмэлт boundary ба error тест 7
2.7 GUI-ийн гар шалгалт 7
2.8 Known issues 7

ХЭСЭГ 1. ХЭРЭГЛЭГЧИЙН ЗААВАР

1.1 Програмын зорилго

Энэ програм нь үйлдлийн системийн тасралтгүй (contiguous) санах ой хуваарилалтыг загварчилна. Та санах ойн хэмжээ (byte) өгч, процессуудад First-fit, Best-fit, эсвэл Worst-fit стратегиар хэсэг олгох (RQ), чөлөөлөх (RL), hole-уудыг шахах (C) болон байдлыг харах (STAT) боломжтой. Санах ой бодит биш, зөвхөн тоогоор бодогдоно. Хоёр интерфейс байна: **CLI** (командын мөр) ба **GUI** (цонх, Memory layout зурагтай). Хоёулаа ижил логикийг ашигладаг тул ижил командад ижил үр дүн гарна.

1.2 Дэмжих орчин

Орчин	Төлөв
Windows 11 + Python 3.11	Шалгасан: мэдэгдэхүйц алдаа гараагүй
Debian / Ubuntu Linux + Python 3.10 ба түүнээс дээш	Ubuntu 24.04 (Python 3.12.3) дээр шалгасан; Debian нь ижил apt багцаар ажиллана
macOS + Python 3.10 ба түүнээс дээш	Шалгаагүй (tkinter байвал ажиллах ёстой)

Шаардлага: Python 3.10 эсвэл түүнээс дээш. Нэмэлт сан суулгахгүй (зөвхөн Python-ийн стандарт сан).

1.3 Суулгах эсвэл нээх алхам

Суулгалт хэрэггүй. Дараах 4 алхмыг дарааллаар хийнэ (ZIP татаж задалсан ч болно):

1. Төслийг татна: `git clone https://github.com/orgilSt1/OS-biydaalt.git`
2. Төслийн фолдерт орно: `cd OS-biydaalt`
3. Python хувилбарыг шалгана: `python --version` (Linux/macOS: `python3 --version`)
4. Зөвхөн Linux дээр GUI ажиллахгүй бол: `sudo apt install python3-tk`
   (Windows-ийн Python суулгагч tkinter-ийг агуулдаг тул нэмэлт зүйл хэрэггүй.)

1.4 Ажиллуулах, зогсоох, reset хийх

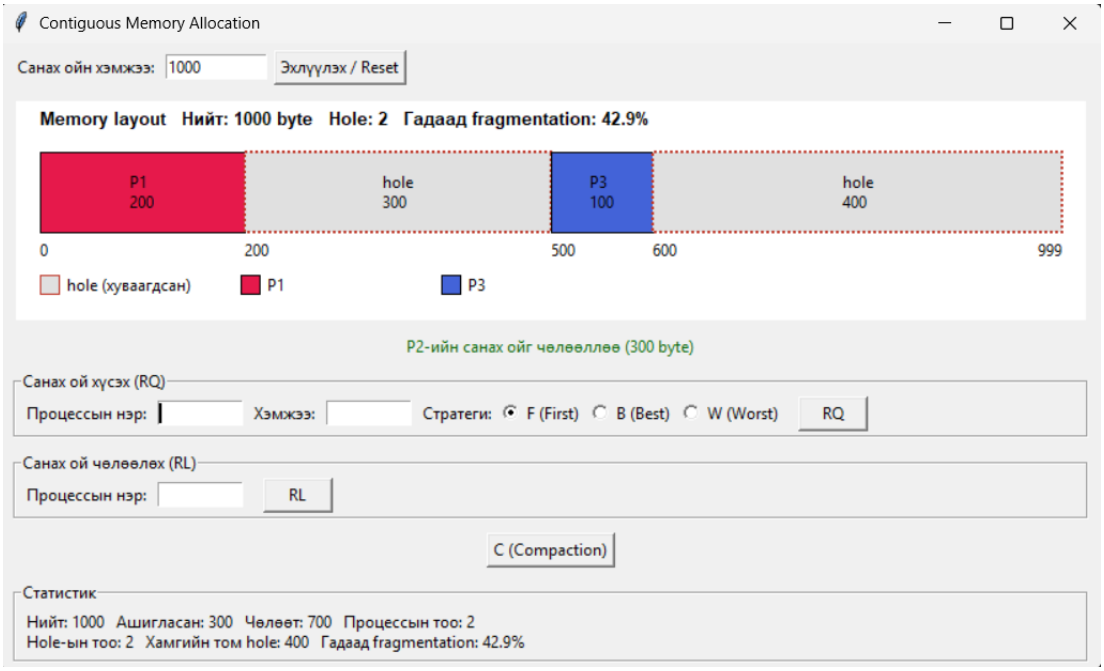
Интерфейс	Ажиллуулах	Зогсоох	Reset
CLI	python src/cli.py 1000 (Linux: python3 src/cli.py 1000) 1000 нь санах ойн хэмжээ, byte	X команд, эсвэл оролтын төгсгөл (Windows: Ctrl+Z, Enter; Linux/macOS: Ctrl+D)	Шууд reset команд байхгүй: X-ээр гараад өөр хэмжээгээр дахин эхлүүлнэ
GUI	python src/gui.py (Linux: python3 src/gui.py)	Цонхыг хаана	Хэмжээний талбарт утга бичиж **Эхлүүлэх** / **Reset** дарна (шинэ хэмжээ ч болно); бүх процесс устана

1.5 Оролт ба тохиргоо

Команд (CLI)	GUI-д	Утга
RQ <нэр> <хэмжээ> <F|B|W>	Процессын нэр, Хэмжээ (byte), Стратеги, **RQ** товч	Нэр: зайгүй (P1). Хэмжээ: 0-ээс их бүхэл тоо. F=First, B=Best, W=Worst
RL <нэр>	Процессын нэр, **RL** товч	Чөлөөлөх процессын нэр; зэргэлдээ hole нэгдэнэ
C	**C (Compaction)** товч	Процессуудыг эхлэл рүү шахаж hole-уудыг нэг болгоно
STAT	Зураг ба статистик автоматаар шинэчлэгдэнэ	Мужуудыг [эхлэх:төгсгөл] хэлбэрээр харуулна
X	Цонхыг хаах	Гарах

Анхаарах: команд том/жижиг үсгээр ялгагдахгүй (rq = RQ), харин процессын нэр ялгагдана (P1 ≠ p1). Бүх хэмжээ, хаяг **byte**-аар. Хаяг нь оролцуулсан: 200 byte-ийн блок 0-ээс эхэлбэл [0:199].

1.6 Дүрслэлийн тэмдэг, өнгө, төлөв (GUI)



Зураг 1. GUI-ийн үндсэн цонх (санах ой 1000 byte, P1 ба P3 хуваарилагдсан, P2 чөлөөлөгдсөн)

Элемент	Утга
Өнгөт тэгш өнцөгт (нэр + хэмжээ дотор нь)	Процессын эзэлсэн блок. Процесс бүр өөр өнгөтэй, өнгө нь процесс чөлөөлөгдсөн ч солигдохгүй
Саарал тэгш өнцөгт ("hole")	Чөлөөт хэсэг
Hole-уудын **улаан тасархай** хүрээ, legend: "hole (хуваагдсан)"	Нэгээс олон hole байна, өөрөөр хэлбэл гадаад fragmentation
Hole-ын **саарал тасархай** хүрээ, legend: "hole (чөлөөт)"	Чөлөөт зай нэг тасралтгүй хэсэг
Блокийн доорх тоо	Блокийн эхлэх хаяг; хамгийн баруун тоо нь сүүлийн хаяг (MAX−1)
Гарчиг: Нийт, Hole, Гадаад fragmentation	Нийт хэмжээ (byte), hole-ын тоо, fragmentation = (чөлөөт − хамгийн том hole) × 100 / чөлөөт
Мессежийн мөр	Ногоон = амжилттай; улаан = алдаа
Статистик	Нийт, Ашигласан, Чөлөөт, Процессын тоо, Hole-ын тоо, Хамгийн том hole, Fragmentation

1.7 Жишээнүүд

Жишээ 1 (CLI): гурван стратегийн ялгаа. Ижил хүсэлтийг F, B, W-ээр өгч, ямар hole сонгогдохыг харна. Командуудыг файлд бичсэн: `examples/example2_fits.txt`.

```
python src\cli.py 1000 < examples\example2_fits.txt
(Linux: python3 src/cli.py 1000 < examples/example2_fits.txt)
```

Эхний STAT-д hole-ууд: [0:199] 200, [250:329] 80, [380:999] 620 byte. Дараа нь 70 byte-ын хүсэлтүүд (тус бүрийн дараа чөлөөлнө):

Команд	Хүлээгдэх гаралт	Ямар hole сонгосон
RQ X 70 F	X-д [0:69] олгов	200-ийн (анхных нь)

Команд	Хүлээгдэх гаралт	Ямар hole сонгосон
RQ Y 70 B	Y-д [250:319] олгов	80-ын (хамгийн жижиг тохирох)
RQ Z 70 W	Z-д [380:449] олгов	620-ийн (хамгийн том)

Жишээ 2 (GUI): fragmentation үүсгэж, compaction-оор засах.

Алхам	Үйлдэл	Харах зүйл
1	Хэмжээ 1000 → **Эхлүүлэх / Reset**	Нэг hole 1000, fragmentation 0%
2	RQ: P1 200 F, P2 300 F, P3 100 F	[0:199] P1, [200:499] P2, [500:599] P3, hole [600:999]
3	RL: P2	Хоёр hole (300 ба 400), улаан тасархай хүрээ, fragmentation 42.9%
4	RQ: P4 250 **B**	P4 нь 300-ийн hole-д [200:449] орно (best-fit), үлдэгдэл 50; fragmentation 11.1%
5	RQ: P5 900 F	Улаан алдаа: санах ой хүрэлцэхгүй (чөлөөт 450 byte, хүссэн 900 byte); төлөв өөрчлөгдөхгүй
6	**C (Compaction)**	P3 [450:549] болж, нэг hole [550:999], саарал тасархай хүрээ, fragmentation 0%

1.8 Түгээмэл алдаа ба шийдэл

Тохиолдол	Шалтгаан	Шийдэл
No module named 'tkinter' / '_tkinter'	Linux дээр tkinter суугаагүй	sudo apt install python3-tk
'python' is not recognized (Windows)	Python PATH-д нэмэгдээгүй	Python-г дахин суулгаж "Add python.exe to PATH"-ийг сонгох. (Linux дээр python олдохгүй бол python3 ашиглана)
Монгол үсэг ? эсвэл эвдэрсэн (Windows)	Terminal-ийн кодчилол UTF-8 биш	Windows Terminal ашиглах, эсвэл chcp 65001 гэж өгөөд дахин ажиллуулах
Хэрэглээ: RQ <нэр> <хэмжээ> <F|B|W>	Аргумент дутуу/илүү, жишээ нь нэрэнд зай ("P 1")	Нэрийг зайгүй бичих: P1
"нэг hole дотор багтахгүй байна (fragmentation)"	Нийт чөлөөт зай хүрэх боловч хуваагдсан	C (Compaction) өгөөд хүсэлтийг давтах
"Эхлээд санах ойн хэмжээг оруулж ... дарна уу" (GUI)	Reset/Эхлүүлэх дараагүй	Хэмжээг бичиж Эхлүүлэх / Reset дарах
Процесс зурагт харагдахгүй (маш жижиг, 1–2 byte)	Блокийн өргөн хэмжээтэй пропорциональ (мэдэгдэж буй хязгаарлалт)	Тоон статистик, legend, STAT-аар шалгах; жижиг санах ойгоор туршиж үзэх
ModuleNotFoundError: allocator	Өөр фолдероос ажиллуулсан эсвэл файл зөөсөн	Төслийн үндсэн фолдероос python src/cli.py ... гэж ажиллуулах; src/ доторх файлуудыг хамт байлгах

ХЭСЭГ 2. OPERATIONAL ЗААВАР

Энэ хэсэг нь шүүмжлэгч баг төслийг өөр компьютер дээр татаж, Build/Run хийж, тестийг давтахад зориулагдсан.

2.1 Repository clone хийх

```
git clone https://github.com/orgilSt1/OS-biydaalt.git
cd OS-biydaalt
```

Фолдерын бүтэц: `src/` (эх код: allocator.py, cli.py, gui.py, gui_canvas.py), `tests/` (test_allocator.py, test_cli.py), `examples/` (example1…4 .txt ба .expected.txt), `docs/` (Manual, Report), `README.md`.

2.2 OS, runtime, version

Python 3.10+ (шалгасан: Windows 11 + Python 3.11; Ubuntu 24.04 + Python 3.12.3). Debian нь Ubuntu-гийн суурь тул ижил python3, python3-tk багцаар ажиллана. Хөрвүүлэгч (compiler) шаардлагагүй: Python нь интерпретатор.

2.3 Dependency

Гадаад сан байхгүй. Зөвхөн Python-ийн стандарт сан: tkinter (GUI), unittest (тест). Linux (Debian/Ubuntu) дээр GUI-д: `sudo apt install python3-tk`.

2.4 Build ба Run команд

Build алхам байхгүй (интерпретатор). Шууд Run хийнэ. Командыг Windows-ийн Command Prompt (cmd) дээр бичсэн, хаалтанд Linux хувилбар:

Зорилго	Команд (төслийн үндсэн фолдероос)	Хүлээгдэх үр дүн
CLI, гар оролт	python src/cli.py 1000 (Linux: python3 src/cli.py 1000)	allocator> prompt гарна
CLI, файлаас	python src\cli.py 1000 < examples\example1.txt (Linux: python3 src/cli.py 1000 < examples/example1.txt)	Командууд давтагдан хэвлэгдэж, STAT-ууд гарна
GUI	python src/gui.py (Linux: python3 src/gui.py)	"Contiguous Memory Allocation" цонх нээгдэнэ
Бүх автомат тест	python -m unittest discover tests (Linux: python3 -m unittest discover tests)	Ran 39 tests ... OK

Тестийг заавал төслийн **үндсэн фолдероос** ажиллуулна. PowerShell дээр < ажилладаггүй тул файлаас оролт өгөхдөө cmd ашиглана, эсвэл `Get-Content examples\example1.txt | python src/cli.py 1000` гэж бичнэ.

2.5 Үндсэн тестийн өгөгдөл

Үндсэн тест: `examples/example1.txt` (санах ой 1000). Гаралтыг хүлээгдэх файлтай харьцуулна:

```
python src\cli.py 1000 < examples\example1.txt > out1.txt
fc out1.txt examples\example1.expected.txt

(Linux: python3 src/cli.py 1000 < examples/example1.txt > out1.txt
        diff out1.txt examples/example1.expected.txt)
```

Зөрүүгүй байх ёстой (fc: "no differences encountered"; Linux-д diff юу ч хэвлэхгүй). Мөрийн төгсгөлийн (CRLF/LF) зөрүүнээс болж fc мэдэгдвэл автомат тестийн *test_examples_match_expected_output* үүнийг шалгана. Гараар бодсон хүлээгдэх STAT:

STAT	Хүлээгдэх муж
1 (RQ P1 200, P2 300, P3 100, F)	[0:199] P1, [200:499] P2, [500:599] P3, [600:999] Unused
2 (RL P1, RL P3)	[0:199] Unused, [200:499] P2, [500:999] Unused
3 (RQ P4 250 B)	[0:199] Unused, [200:499] P2, [500:749] P4, [750:999] Unused
4 (C)	[0:299] P2, [300:549] P4, [550:999] Unused

Номын жишээ (1 MB): `python src/cli.py 1048576` (Linux: `python3 src/cli.py 1048576`) дээр `RQ P0 40000 W` өгвөл `P0`-д `[0:39999]` олгов, `STAT` нь `Addresses [0:39999] Process P0` ба `Addresses [40000:1048575] Unused` гарна.

2.6 Нэмэлт boundary ба error тест

Тест	Ролт	Хүлээгдэх үр дүн
Буруу эхлүүлэх аргумент	python src/cli.py (аргументгүй); ... abc; ... 0; ... -5 (Linux: python3)	Аргументгүй бол "Хэрэглээ: python src/cli.py <санах ойн хэмжээ (byte)>"; abc, 0, -5 бол "Алдаа: санах ойн хэмжээ 0-ээс их бүхэл тоо байх ёстой"
1 byte-ийн санах ой	python src/cli.py 1 (Linux: python3 src/cli.py 1), дараа нь RQ A 1 F, RQ B 1 F	A-д [0:0] олгов; B: Алдаа: санах ой хүрэлцэхгүй (чөлөөт 0 byte, хүссэн 1 byte)
Яг тэнцүү хэмжээ	Санах ой 100: RQ a 100 F; дараа нь RQ b 1 B	a-д [0:99] олгов; үлдэгдэл hole үүсэхгүй; b хүрэлцэхгүй
Давхар чөлөөлөлт	RL A хоёр удаа	Эхнийх амжилттай; хоёр дахь: Алдаа: A нэртэй процесс олдсонгүй
Алдаатай командууд	examples/example4_errors.txt	Бүх мөрд "Алдаа" эсвэл "Хэрэглээ" гарна, программ унахгүй (Report-ын 8.6-р хэсэг)
Fragmentation ба compaction	examples/example3_compaction.txt	RQ E 350 F эхэндээ алдаа (нийт 400 чөлөөтэй ч багтахгүй); C-ийн дараа E-д [600:949] олгов
Том/жижиг үсэг	rq a 100 f	Ажиллана (команд ялгагдахгүй), нэр a хэвээр

2.7 GUI-ийн гар шалгалт

python src/gui.py (Linux: python3 src/gui.py) ажиллуулаад Хэсэг 1.7-ийн Жишээ 2-ыг давтана. Нэмэлтээр: хоосон нэр, "P 1", хэмжээ abc / 0 / −5 / хэт том утга, байхгүй процессыг RL, Reset-д abc эсвэл 0 өгөх. Бүх тохиолдолд цонх унахгүй, ойлгомжтой мессеж (улаан эсвэл алдааны цонх) гарна.

2.8 Known issues

1–2 byte-ийн хэт жижиг блок зурагт харагддаггүй (Report-ын 9.3-р хэсэг). GUI-д командын түүх, undo байхгүй. Процессын нэр том/жижиг үсгээр ялгагдана. Хүрэлцэхгүй хүсэлтийг хүлээлгийн дарааламд тавихгүй, татгалзана. Windows 11 дээр мэдэгдэхүйц алдаа гараагүй.